

Решетчатое представление кода и построение решетки по порождающей матрице

1. Решетка кода как способ описания всех кодовых слов

Для двоичного кода длины n можно построить решетку — ориентированный граф с $n + 1$ ярусами, в котором:

- нулевой ярус соответствует моменту до чтения первого символа;
- i -й ярус соответствует тому, что уже обработаны первые i символов кодового слова;
- ребро между ярусами $i - 1$ и i помечается i -м кодовым символом;
- любой путь из начального узла в конечный задает ровно одно кодовое слово;
- каждое кодовое слово кода должно быть реализовано некоторым путем в решетке.

Если длина кода равна n , то число ярусов равно $n + 1$. Начальный узел один, конечный узел тоже один. Внутренние ярусы содержат промежуточные состояния, соответствующие частично пройденному пути.

Идея решетчатого представления состоит в том, что кодовое слово рассматривается не целиком, а как последовательность локальных переходов. За счет этого задача декодирования заменяется задачей поиска наилучшего пути в графе.

2. Общее построение решетки по множеству кодовых слов

Этот способ не использует линейность кода и применим вообще к любому конечному коду.

Пусть дан код

$$C = \{c^{(1)}, c^{(2)}, \dots, c^{(M)}\}, \quad c^{(m)} = (c_1^{(m)}, \dots, c_n^{(m)}).$$

На каждом ярусе кодовое слово разбивается на две части:

$$(c_1, \dots, c_i) \mid (c_{i+1}, \dots, c_n).$$

Первая часть интерпретируется как **прошлое**, вторая — как **будущее**. Построение можно понимать так:

1. На нулевом ярусе прошлое пусто, а будущее совпадает со всем множеством кодовых слов.
2. На первом ярусе слова группируются по первому символу.
3. На втором ярусе — по первым двум символам.
4. И так далее, пока не будут исчерпаны все символы.

Однако простого разбиения по префиксам недостаточно. Если на некотором ярусе разные пройденные фрагменты приводят к одному и тому же множеству допустимых будущих продолжений, такие пути нужно **склеить** в один узел. Именно эта операция и приводит к компактной решетке.

Поэтому узел на ярусе фактически описывает не только уже прочитанное прошлое, но и то, какие продолжения остаются возможными дальше.

3. Структура узлов и ребер

Для яруса i каждый узел соответствует классу эквивалентных частичных путей. Эквивалентность можно описывать двумя взаимодополняющими способами:

- либо у путей совпадает допустимое множество будущих продолжений;
- либо у кодовых слов, проходящих через этот узел, совпадает уже накопленное прошлое в смысле дальнейшего поведения решетки.

Переход между соседними ярусами помечается кодовым символом $c_i \in \{0, 1\}$. Поэтому путь

$$e_1, e_2, \dots, e_n$$

однозначно порождает кодовое слово

$$c = (c_1, c_2, \dots, c_n),$$

где c_i — метка ребра между ярусами $i - 1$ и i .

В правильной решетке выполняются два свойства:

1. каждому кодовому слову соответствует путь из начального узла в конечный;
2. никакой лишний путь не должен порождать последовательность, не принадлежащую коду.

4. Решетка как инструмент декодирования

После построения решетки задача декодирования превращается в задачу поиска наилучшего пути.

Пусть по двоичному симметричному каналу принято слово

$$y = (y_1, \dots, y_n).$$

Для ребра, на котором лежит символ c_i , вводится локальная метрика. Для канала с жестким решением естественно брать расстояние Хэмминга между принятым символом и символом на ребре:

$$\mu_i = d_H(y_i, c_i) = \begin{cases} 0, & y_i = c_i, \\ 1, & y_i \neq c_i. \end{cases}$$

Метрика пути равна сумме локальных метрик:

$$\mu(c | y) = \sum_{i=1}^n d_H(y_i, c_i) = d_H(y, c).$$

Следовательно, декодирование по максимуму правдоподобия в двоичном симметричном канале сводится к поиску пути минимальной суммарной метрики:

$$\hat{c} = \arg \min_{c \in C} d_H(y, c).$$

Это и есть решетчатое декодирование по критерию минимального расстояния.

5. Накопление метрики и отсеечение невыгодных путей

При движении слева направо на каждом ярусе метрика на новом узле вычисляется как минимум по всем входящим путям:

$$M_i(v) = \min_{u \rightarrow v} (M_{i-1}(u) + \mu_i(u \rightarrow v)),$$

где $M_i(v)$ — лучшая накопленная метрика к узлу v на ярусе i .

Если в один и тот же узел приходят несколько путей, то сохраняется только путь с наименьшей накопленной метрикой. Все остальные пути можно отбросить, потому что при одинаковом будущем они уже не смогут стать лучше лидирующего пути. Именно эта идея лежит в основе динамического декодирования по решетке.

Таким образом, на каждом ярусе происходит:

1. добавление локальной метрики ребра;
2. сравнение конкурирующих путей, пришедших в один узел;
3. сохранение только лучшего представителя для каждого узла.

После достижения конечного узла восстанавливается путь минимальной метрики, а вместе с ним и декодированное кодовое слово.

6. Профиль сложности решетки

Решетка характеризуется **профилем сложности** — числом узлов на каждом ярусе. Если на ярусах $0, 1, \dots, n$ находится соответственно

$$\xi_0, \xi_1, \dots, \xi_n$$

узлов, то профиль записывается как

$$(\xi_0, \xi_1, \dots, \xi_n).$$

Чем больше число узлов на промежуточных ярусах, тем выше вычислительная сложность декодирования. Поэтому важно строить как можно более компактную решетку.

7. Минимальная решетка

Решетка называется **минимальной**, если на каждом ярусе не осталось лишних различных состояний.

Практический критерий минимальности:

На каждом ярусе все пути, имеющие одинаковое будущее или одинаковое прошлое, проходят через один и тот же узел.

Иначе говоря, если два частичных пути в дальнейшем неразличимы, их нельзя держать в разных узлах: они должны быть слиты. Для любого кода минимальная решетка существует; различаться она может только переименованием узлов.

8. Почему прямое построение по всем кодовым словам неудобно

Если строить решетку напрямую по множеству всех кодовых слов, то сначала нужно пересчитать весь код. Для линейного (n, k) -кода это означает работу с 2^k словами. Поэтому прямой метод становится экспоненциальным по размеру информационного пространства.

Для линейных кодов можно использовать структуру порождающей матрицы и получать ту же решетку существенно более конструктивно. Именно для этого вводятся понятия начала, конца и *span* строки.

9. Начало, конец и *span* двоичного вектора

Пусть

$$x = (x_1, x_2, \dots, x_n) \in \mathbb{F}_2^n.$$

Определяются следующие величины:

- **начало** вектора

$$b(x) = \min\{i \mid x_i \neq 0\};$$

- **конец** вектора

$$e(x) = \max\{i \mid x_i \neq 0\};$$

- **span** вектора — отрезок позиций от первого ненулевого элемента до последнего ненулевого элемента;
- **длина span**

$$\text{span}(x) = e(x) - b(x) + 1.$$

Если, например,

$$x = (0, 0, 1, 1, 1, 0, 1),$$

то

$$b(x) = 3, \quad e(x) = 7,$$

а активные позиции лежат на отрезке от 3 до 7. Длина span равна

$$7 - 3 + 1 = 5.$$

При построении решетки по порождающей матрице особенно важны строки, которые **активны** на некотором ярусе. Если строка начинается не позже данного яруса, но заканчивается позже него, то она еще влияет на дальнейшее развитие пути.

Формально строка $g^{(j)}$ считается активной на ярусе i , если

$$b(g^{(j)}) \leq i < e(g^{(j)}).$$

Именно такое определение соответствует тому, что последний ненулевой символ строки уже не создает ветвления дальше: после него влияние строки на будущие ярусы прекращается.

10. Минимальная span-форма порождающей матрицы

Пусть линейный код задан порождающей матрицей

$$G = \begin{pmatrix} g^{(1)} \\ g^{(2)} \\ \vdots \\ g^{(k)} \end{pmatrix}.$$

Матрица называется приведенной к **минимальной span-форме**, если:

- начала всех строк различны;
- концы всех строк различны.

То есть ни у каких двух строк не совпадает позиция первого ненулевого элемента и ни у каких двух строк не совпадает позиция последнего ненулевого элемента.

Такую форму можно получать элементарными строковыми преобразованиями, не меняющими код:

$$g^{(i)} \leftarrow g^{(i)} + g^{(j)}.$$

Поскольку строки складываются по модулю 2, код как линейное пространство не меняется, но расположение единиц в строках можно сделать более удобным для решетчатого построения.

11. Матрица примера в минимальной span-форме

В лекционном примере после преобразований используется матрица

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Это бинарный линейный код длины 6 и размерности 3.

Для строк этой матрицы имеем:

$$g^{(1)} = (1, 1, 0, 1, 0, 0), \quad b_1 = 1, \quad e_1 = 4,$$

$$g^{(2)} = (0, 1, 1, 0, 1, 0), \quad b_2 = 2, \quad e_2 = 5,$$

$$g^{(3)} = (0, 0, 0, 1, 1, 1), \quad b_3 = 4, \quad e_3 = 6.$$

Начала строк различны: 1, 2, 4. Концы строк тоже различны: 4, 5, 6. Следовательно, матрица действительно находится в минимальной span-форме.

Активные строки по ярусам:

ярус	активные строки
0	$\emptyset$
1	$\{1\}$
2	$\{1, 2\}$
3	$\{1, 2\}$
4	$\{2, 3\}$
5	$\{3\}$
6	$\emptyset$

Число активных строк равно соответственно

$$0, 1, 2, 2, 2, 1, 0.$$

Поэтому число узлов на ярусах равно

$$2^0, 2^1, 2^2, 2^2, 2^2, 2^1, 2^0,$$

то есть профиль решетки имеет вид

$$(1, 2, 4, 4, 4, 2, 1).$$

12. Кодовые слова примера

Пусть информационный вектор

$$u = (u_1, u_2, u_3) \in \mathbb{F}_2^3.$$

Тогда кодовое слово вычисляется как

$$c = uG.$$

Все $2^3 = 8$ кодовых слов данного кода:

$$\begin{aligned} 000 &\mapsto 000000, \\ 001 &\mapsto 000111, \\ 010 &\mapsto 011010, \\ 011 &\mapsto 011101, \\ 100 &\mapsto 110100, \\ 101 &\mapsto 110011, \\ 110 &\mapsto 101110, \\ 111 &\mapsto 101001. \end{aligned}$$

Этот список совпадает с путями, которые должна реализовывать решетка.

13. Правило нумерации узлов по активным информационным символам

Пусть на ярусе i активны строки с номерами из множества

$$A_i = \{j \mid b_j \leq i < e_j\}.$$

Тогда узлы яруса i нумеруются значениями информационных символов

$$(u_j)_{j \in A_i}.$$

Иными словами, состояние на ярусе определяется именно теми информационными символами, которые еще продолжают влиять на будущую часть кодового слова.

Для матрицы примера это дает:

- ярус 0: активных строк нет, поэтому один узел;
- ярус 1: активна только первая строка, поэтому узлы 0 и 1;
- ярусы 2 и 3: активны строки 1 и 2, поэтому узлы 00, 01, 10, 11;
- ярус 4: активны строки 2 и 3, поэтому снова узлы 00, 01, 10, 11;
- ярус 5: активна только третья строка, поэтому узлы 0 и 1;
- ярус 6: снова один конечный узел.

Таким образом, профиль $(1, 2, 4, 4, 4, 2, 1)$ получается непосредственно из структуры активных строк матрицы.

14. Правило вычисления меток на ребрах

Ребро между ярусами $i - 1$ и i соответствует i -му кодовому символу. Если в i -м столбце матрицы G стоят единицы в строках с номерами $j_1, \dots, j_s$, то метка ребра вычисляется как

$$c_i = u_{j_1} + u_{j_2} + \dots + u_{j_s} \pmod{2}.$$

Иначе говоря, кодовый символ есть линейная комбинация тех информационных символов, чьи строки активны и содержат единицу в данном столбце.

Для примера:

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}$$

дает столбцы

$$\begin{aligned} c_1 &= u_1, \\ c_2 &= u_1 + u_2, \\ c_3 &= u_2, \\ c_4 &= u_1 + u_3, \\ c_5 &= u_2 + u_3, \\ c_6 &= u_3. \end{aligned}$$

Отсюда сразу читается вся структура ребер между соседними ярусами.

15. Как строятся переходы между узлами

Переход между узлами соседних ярусов разрешен тогда и только тогда, когда значения информационных символов, общих для двух соседних ярусов, согласованы между собой.

Например:

- между ярусами 1 и 2 должен сохраняться символ u_1 ;
- между ярусами 2 и 3 сохраняются оба символа u_1, u_2 ;
- между ярусами 3 и 4 символ u_1 перестает быть активным, а вместо него активируется u_3 , поэтому переход связывает состояние (u_1, u_2) с состоянием (u_2, u_3) ;
- между ярусами 4 и 5 должен сохраняться символ u_3 .

Именно поэтому решетка не строится произвольно: ее структура полностью задается тем, какие информационные символы активны на соседних ярусах и как вычисляется очередной кодовый символ.

16. Декодирование на решетке для двоичного симметричного канала

Пусть принято слово

$$y = (y_1, \dots, y_6).$$

Для каждого ребра вычисляется локальная метрика

$$\mu_i = d_H(y_i, c_i),$$

после чего на каждом узле сохраняется только путь с наименьшей суммарной метрикой. Если в узел приходят два пути со стоимостью 2 и 3, то путь с метрикой 3 немедленно отбрасывается, потому что дальнейшая часть пути для обоих будет проходить через одно и то же состояние и уже не сможет компенсировать проигрыш на единицу.

На последнем ярусе остается путь минимального веса. Он задает ближайшее к принятому слову кодовое слово.

Для двоичного симметричного канала это эквивалентно декодированию по максимуму правдоподобия, поскольку наиболее вероятным является кодовое слово с минимальным расстоянием Хэмминга до принятого вектора.

17. Секционная решетка

Если на некотором участке решетки внутренний ярус не создает новой существенной неопределенности, несколько соседних ярусов можно объединить в один участок. Тогда на ребре лежит не один кодовый символ, а сразу блок символов.

Такая решетка называется **секционной**. Она сохраняет тот же код, но меняет профиль сложности, уменьшая число промежуточных узлов. В лекционном примере после такого склеивания профиль становится

$$(1, 2, 4, 4, 2, 1),$$

то есть один из промежуточных ярусов устраняется, а на части ребер появляются двухсимвольные метки.

Смысл секционной решетки в том, что она сжимает участки, где структура путей уже однозначно определяется предыдущим состоянием.

18. Почему решетка из матрицы в минимальной span-форме минимальна

Нужно показать, что на любом ярусе разные состояния действительно соответствуют различным будущим и потому не могут быть склеены.

Рассмотрим некоторый ярус l и множество строк, активных на этом ярусе:

$$A_l = \{j \mid b_j \leq l < e_j\}.$$

Состояние узла задается набором значений

$$(u_j)_{j \in A_l}.$$

Предположим, что имеются два различных набора значений активных информационных символов, но при этом они якобы дают одно и то же будущее. Тогда их разность порождает нетривиальную линейную комбинацию активных строк, у которой вся правая часть после яруса l должна была бы обратиться в нуль.

Покажем, что это невозможно.

Выберем в этой нетривиальной линейной комбинации строку с максимально правым концом. Так как в минимальной span-форме концы всех строк различны, то в позиции этого максимального конца только выбранная строка имеет единицу, а все остальные строки комбинации в этой позиции уже равны нулю. Следовательно, сумма в этой позиции не может обнулиться.

Значит, любая нетривиальная линейная комбинация активных строк обязательно имеет ненулевой символ в некоторой позиции правее l . Следовательно, различные наборы активных информационных символов порождают различные будущие продолжения.

Отсюда следует, что два пути с различными состояниями на ярусе l не могут иметь одно и то же будущее. Аналогично, за счет различия начал строк не возникает отождествления различных прошлых. Следовательно, построенная решетка минимальна.

19. Итоговая схема построения решетки по порождающей матрице

Для линейного кода процедура имеет следующий вид.

1. Привести порождающую матрицу к минимальной span-форме.
2. Для каждого яруса определить множество активных строк.
3. Число узлов на ярусе взять равным числу возможных наборов активных информационных символов:

$$|V_i| = 2^{|A_i|}.$$

4. Узлы пронумеровать значениями этих активных символов.
5. Между соседними ярусами провести совместимые переходы.
6. Метку ребра вычислять как значение соответствующего кодового символа, то есть как линейную комбинацию единиц в соответствующем столбце матрицы.
7. После построения решетки выполнять декодирование как поиск пути минимальной суммарной метрики.

20. Основные выводы

- Решетка описывает код как множество путей в ориентированном графе.
- Общее построение по всем кодовым словам применимо и к нелинейным кодам, но требует перебора всего кода.
- Для линейных кодов решетку можно строить напрямую по порождающей матрице.

- Ключевыми понятиями являются начало строки, конец строки, span и активные строки.
- Минимальная span-форма порождающей матрицы обеспечивает минимальность построенной решетки.
- Число узлов на ярусе определяется числом активных информационных символов.
- Решетчатое декодирование по метрике Хэмминга реализует декодирование по максимуму правдоподобия в двоичном симметричном канале.
- Секционная решетка уменьшает профиль сложности за счет объединения нескольких соседних ярусов.